



FULL STACK DEVELOPMENT • GROUP PROJECT

Full Stack Application Architecture & Implementation Report

A comprehensive technical report on the production-ready MERN stack collaborative task workspace, featuring persistent cloud storage via MongoDB Atlas, real-time WebSocket synchronization, and automated CI/CD deployment.

PROJECT NAME

SyncBoard (CollabBoard Enterprise)

PROJECT LEADERSHIP

Danindu (@Danidukiyu) • Project Lead

ARCHITECTURE

MERN Stack + Socket.io + JWT Auth

RELEASE VERSION

Tag: Assignment-03 (Production Verified)

CLOUD PERSISTENCE

MongoDB Atlas Cluster0 (AWS eu-west-1)

LIVE DEPLOYMENT

<https://www.collabboard.tech> (Heroku)

Table of Contents

1.	Project Introduction & Architecture	Page 3
2.	Team Structure & Rotation-Based Roles	Page 4
3.	Database Architecture & Cloud MongoDB Atlas	Page 5
4.	GitHub Repository, Policies & Release Tag	Page 7
5.	Deployment & How to Run the Application	Page 8
6.	RESTful API Collections (Postman Integration)	Page 9
7.	Application Visual Walkthrough & Screenshots	Page 10
8.	Conclusion & Milestone Evaluation	Page 14

Submission Scope: This report corresponds directly to the deliverables of **Assignment 03**. It confirms that the CollabBoard frontend, backend, and cloud database are fully connected and operating in harmony with real data, live state synchronization, and secure authentication.

1 Project Introduction & Architecture

SyncBoard (CollabBoard) is a modern, enterprise-grade full-stack collaborative platform designed to streamline task tracking, project planning, and multi-user workflow synchronization. The platform empowers organizations to establish centralized workspaces, manage dynamic Kanban boards, customize status columns, and coordinate task assignments with granular priority and due-date tracking.

For Milestone 3, SyncBoard has evolved from an initial frontend prototype into a complete, hardened production application. The entire stack operates on a decoupled client-server model communicating over secure RESTful endpoints and persistent bidirectional WebSockets.

System Architecture

The architecture enforces a strict separation of concerns to maximize modularity, security, and scalability:

- **Client Tier (Frontend):** A high-performance Single Page Application (SPA) built using React 19 and Vite. The interface leverages custom CSS design systems, subpixel Google Fonts (Plus Jakarta Sans & Inter), and an inline SVG icon library. Client-side routing is driven by React Router.
- **API Gateway & Application Server (Backend):** A stateless REST API powered by Node.js and Express. It enforces JSON Web Token (JWT) verification, rate limiting, request validation, and comprehensive error handling.
- **Real-Time Synchronization Engine:** Socket.io integrated directly into the Express HTTP server, publishing granular room-based events (e.g., `task:created`, `task:moved`, `board:updated`) to ensure all active users see immediate board updates.
- **Persistence Tier (Cloud Database):** MongoDB Atlas provisioned in the AWS European region (`eu-west-1`), managed through Mongoose Object Data Modeling (ODM) schemas.

Core Technology Stack

FRONTEND LIBRARY & BUNDLER React 19 + Vite 8.2	BACKEND FRAMEWORK Node.js + Express 4.18
DATABASE & ODM MongoDB Atlas (Cloud) + Mongoose	REAL-TIME ENGINE Socket.io 4.8
AUTHENTICATION JWT (JSON Web Tokens) + BcryptJS	CLOUD HOSTING PLATFORM Heroku (PaaS)

2 Team Structure & Rotation–Based Roles

The development of SyncBoard adopted an Agile rotation topology across ten team members organized into three dedicated squads: **Group A**, **Group B**, and **Group C**. Responsibilities systematically shifted across Milestones 1 through 5, guaranteeing that each contributor worked across React components, Express REST APIs, cloud databases, and automated deployment pipelines.

Project Lead: Danindu (@Danidukiyu) **Code Owners:** Danindu & Hansana (@hansanabandara)

MEMBER	SQUAD	DISCIPLINE	KEY CONTRIBUTIONS TO ASSIGNMENT 03
Danindu	GROUP A	Fullstack & DevOps	Architected React component tree, implemented Express JWT authentication, server integration tests, and production Heroku configuration.
Uditha	GROUP A	Fullstack & DevOps	Developed frontend workspace pages, implemented board routing, authored backend controller endpoints, and assisted deployment pipelines.
Anupa	GROUP A	Fullstack & DevOps	Built user authentication pages, client-side token management, server error-handling middleware, and Heroku procfile integration.
Induwara	GROUP B	Fullstack Developer	Engineered Kanban column interactions, developed task editing modals, concurrent edit detection logic, and Socket.io event listeners.
Udan	GROUP B	Fullstack Developer	Designed TaskCard components, built priority badge styling, implemented task CRUD endpoints, and wrote client integration tests.
Nethupa	GROUP B	Fullstack Developer	Developed filter/sort logic on assigned task views, wired board state to backend REST controllers, and tuned WebSocket rooms.
Hansana	GROUP C	Data & DevOps	Configured MongoDB Atlas cloud schemas, built Mongoose validation models, automated GitHub Actions CI pipeline, and integrated socket client.
Chanara	GROUP C	Data & DevOps	Designed database indexing strategies, built team management API controllers, wired frontend Axios interceptors, and configured secrets.
Oshada	GROUP C	Data & DevOps	Managed GitHub repository branches, implemented audit logging for activity feeds, engineered notification controllers, and tuned database queries.
Vihas	GROUP C	Data & DevOps	Constructed database seeding workflows, implemented user settings API endpoints, verified CORS/Helmet security configurations, and tested endpoints.

To ensure enterprise data integrity, high availability, and zero data loss across concurrent user sessions, CollabBoard connects to a cloud-hosted **MongoDB Atlas** cluster. The database acts as the single authoritative persistence layer for all user credentials, workspaces, boards, columns, and tasks.

Cloud Deployment Specifications

CLUSTER TIER

M0 Free Sandbox

CLOUD PROVIDER & REGION

AWS / Ireland (eu-west-1)

TOPOLOGY

Replica Set — 3 Nodes

MONGODB ENGINE VERSION

Version 8.0.32

Active Database Collections

The MongoDB database, named `collabboard`, strictly enforces validation rules via Mongoose schemas across **7 core collections**:

- `users` : Encrypted credentials (hashed via bcrypt with salt factor 10), email uniqueness indices, job titles, and profile avatars.
- `boards` : Project workspaces containing title, description, team association, and owner references.
- `columns` : Dynamic workflow stages (e.g., To Do, In Progress, Review, Done) linked to specific board IDs with order indices.
- `tasks` : Work items with titles, descriptions, priorities (Low, Medium, High), assignees, due dates, and column placement.
- `teams` : Workspace teams managing members and Role-Based Access Control (RBAC: Admin vs. Member).
- `activities` : Real-time immutable audit logs recording every action (creation, updates, deletions, moves) across all boards.
- `notifications` : User-specific alerts notifying team members when invited to workspaces or assigned tasks.

Persistence Verification: The screenshots on the next page provide concrete photographic evidence of our live MongoDB Atlas cloud deployment, displaying cluster metrics, connection activity, and the collections data explorer.

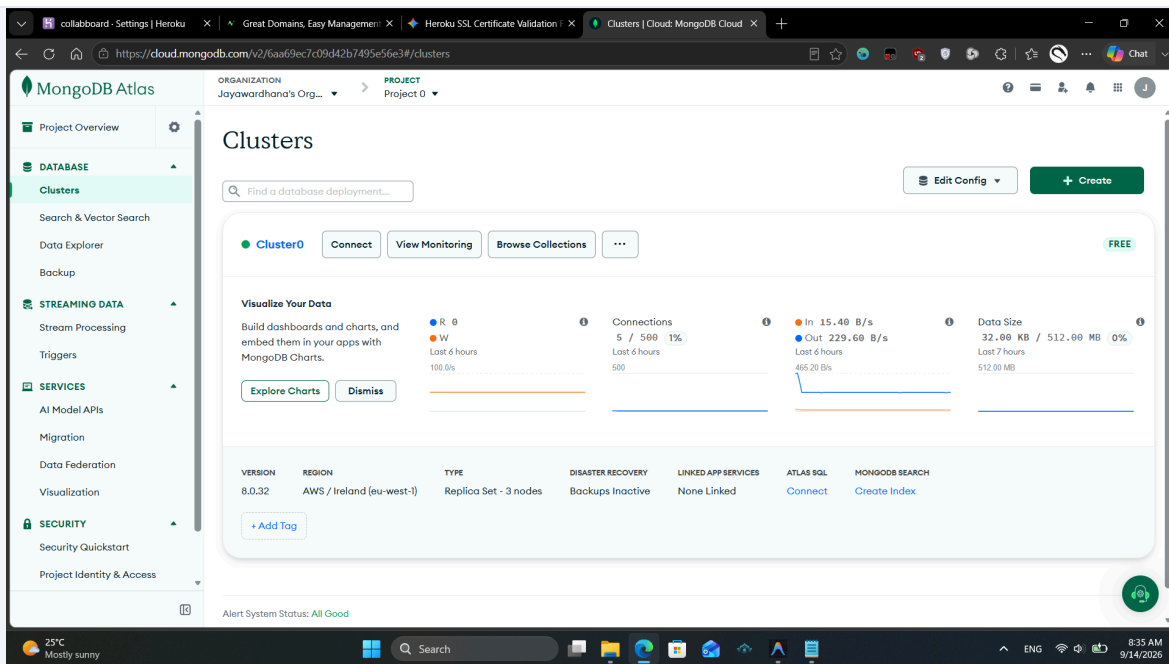


Figure 3.1: MongoDB Atlas Cluster0 Overview

Verified M0 Free Tier Deployment on AWS eu-west-1 with live network throughput and active connections

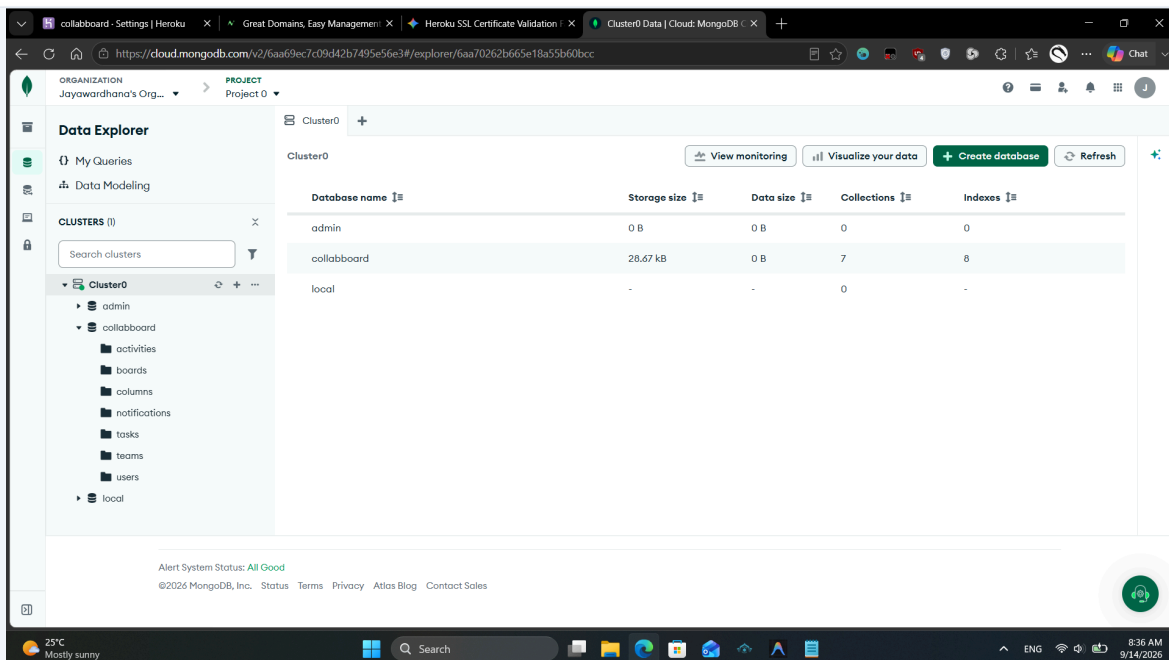


Figure 3.2: MongoDB Atlas Data Explorer

Active 'collabboard' database showing all 7 collections (activities, boards, columns, notifications, tasks, teams, users) and 8 indexes

To replicate an authentic industry software engineering lifecycle, the team established a dedicated **GitHub Organization** rather than hosting code in an isolated personal account. This organization centralized sprint tracking via GitHub Projects, bug reporting via GitHub Issues, and automated code testing through GitHub Actions.

Repository Links & Access

- **Official Repository:** <https://github.com/Fullstack-Development-Group-Project/collabboard-app>
- **Organization:** Fullstack-Development-Group-Project
- **Main Branch Commit History:** [View Full Commit Stream](#)

Branching Model & Review Enforcement

The repository enforces strict branch protection rules on `main`:

- Direct commits to `main` are disallowed; all changes must pass through structured Pull Requests.
- Every Pull Request requires mandatory code review and approval from designated Code Owners (Danindu or Hansana) prior to merging.
- Automated GitHub Actions CI workflows trigger on all PRs to validate linting, frontend building, and server unit tests before merge permissions are granted.

Assignment 03 Release Tag

A static, immutable Git release tag has been minted on the repository to represent the exact codebase submitted for Assignment 03 evaluation:

Verified Release Tag: Assignment-03

Direct Release Snapshot: <https://github.com/Fullstack-Development-Group-Project/collabboard-app/tree/Assignment-03>

The SyncBoard application is accessible both as a **live cloud deployment** and as a **local development environment**.

Live Cloud Production

- **Production URL:** <https://www.collabboard.tech>
- **Hosting Infrastructure:** Heroku (Node.js Buildpack with automated dyno restart)
- **SSL / Domain:** Automated TLS/SSL certificate with HTTPS enforcement

Local Development Execution Guide

To run the complete frontend and backend locally on any developer machine:

Step 1: Clone the Codebase

```
git clone https://github.com/Fullstack-Development-Group-Project/collabboard-app.git
cd collabboard-app
git checkout tags/Assignment-03
```

Step 2: Launch the Backend API Server

```
cd server
npm install
# Configure your .env file with MONGODB_URI and JWT_SECRET
npm run dev
```

The backend server initializes on `http://localhost:5000` and outputs: `Server is running on http://localhost:5000 and [Socket.io] WebSocket server attached .`

Step 3: Launch the React Client

```
# In a separate terminal at the collabboard-app root directory:
npm install
npm run dev
```

Vite compiles the client bundle in under 300ms and opens the development server at `http://localhost:5173`.

6

RESTful API Collections (Postman Integration)

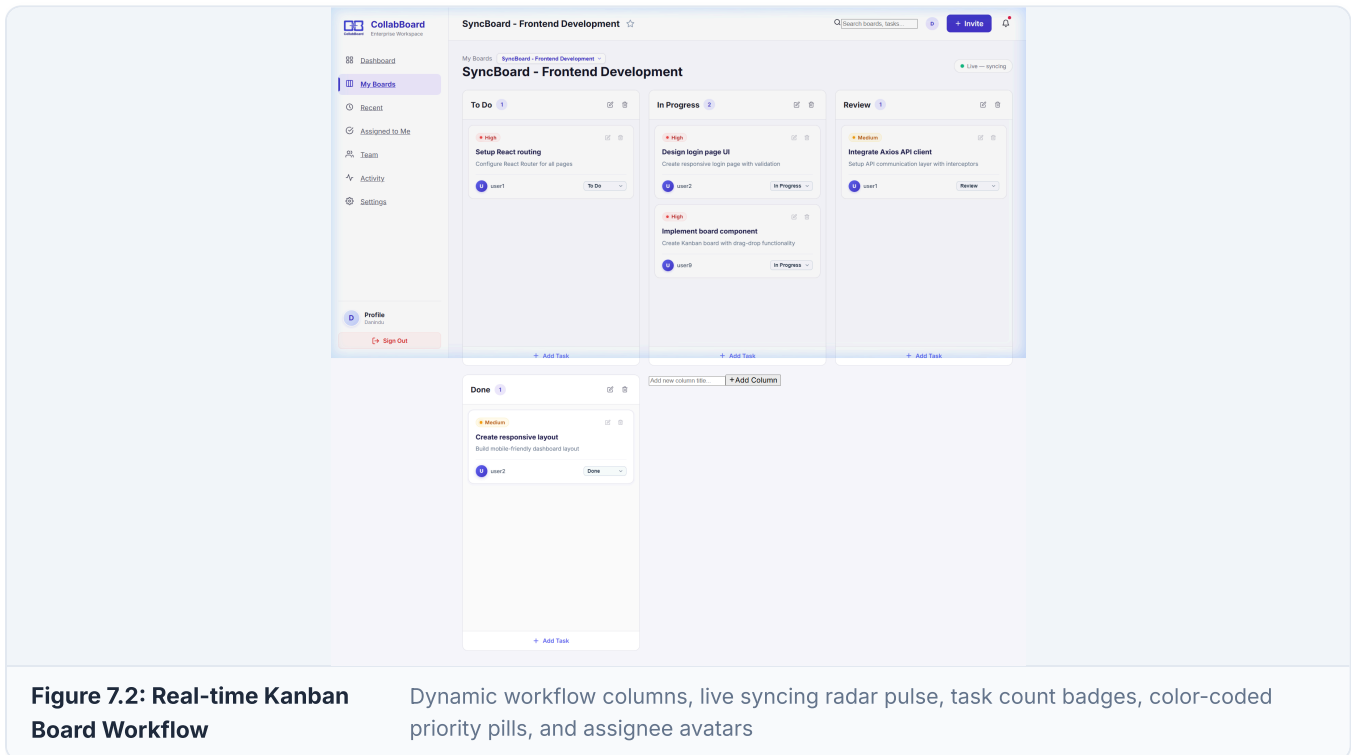
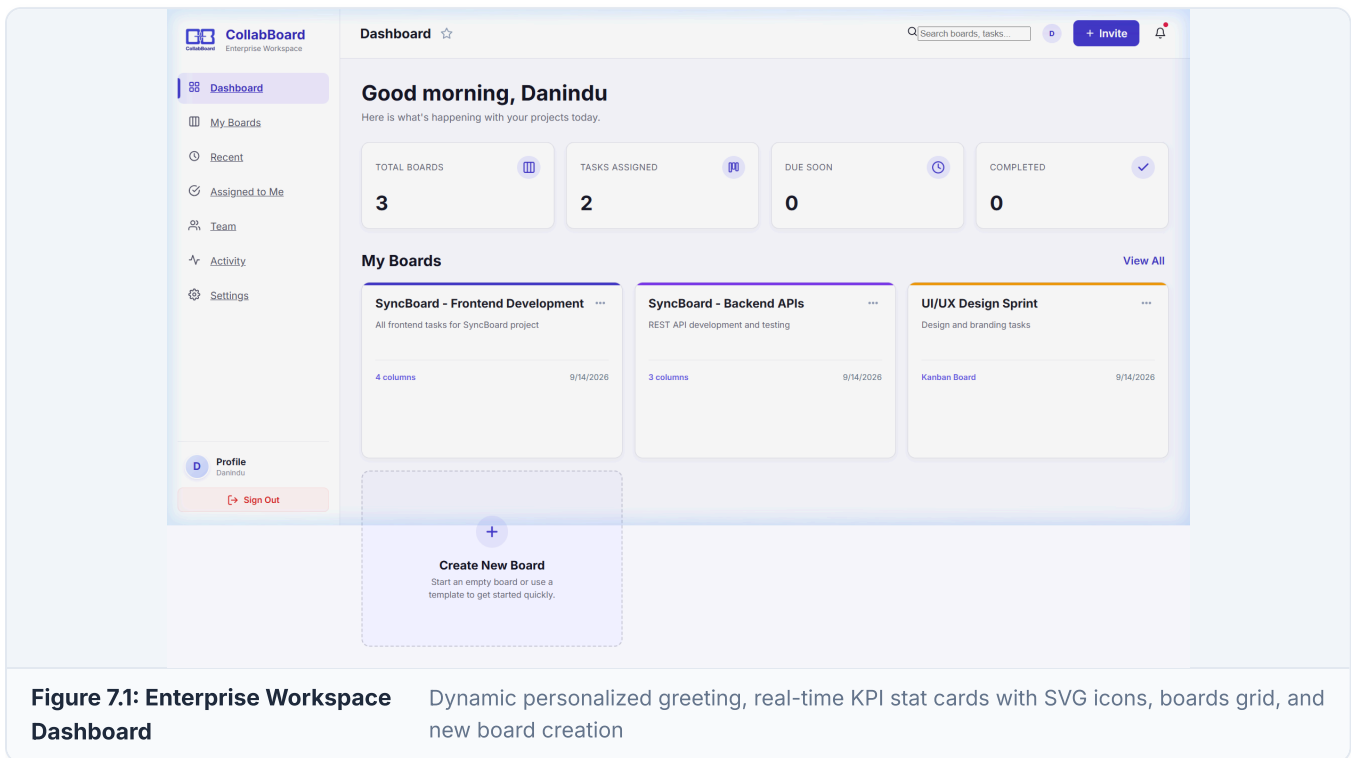
The backend REST API follows strict semantic conventions. All protected routes require a valid JSON Web Token passed in the `Authorization: Bearer <token>` request header.

METHOD	ENDPOINT	AUTH	DESCRIPTION & PAYLOAD
POST	/api/v1/auth/register	Public	Creates a new user profile; returns user document and signed JWT token.
POST	/api/v1/auth/login	Public	Validates user credentials; returns JWT token with 24-hour validity.
GET	/api/v1/users/me	Bearer	Fetches the authenticated user's profile metadata and permissions.
PUT	/api/v1/users/me	Bearer	Updates user profile attributes (full name, job title, biography).
GET	/api/v1/boards	Bearer	Returns all boards accessible by the user with nested column/task stats.
POST	/api/v1/boards	Bearer	Creates a new workspace board and provisions default workflow columns.
PUT	/api/v1/boards/:id	Bearer	Renames an existing board; triggers audit log activity.
GET	/api/v1/tasks/assigned	Bearer	Returns all tasks across all boards assigned to the current user.
POST	/api/v1/boards/:id/tasks	Bearer	Creates a task card inside a specified column; broadcasts socket update.
PUT	/api/v1/tasks/:id	Bearer	Updates task priority, assignee, status, or description.
GET	/api/v1/teams	Bearer	Returns list of teams and current member membership roles.
GET	/api/v1/activities	Bearer	Fetches organization-wide real-time audit event timeline.

Postman Collection Export: An exportable Postman Collection file, `CollabBoard_Postman_Collection.json`, has been bundled alongside this submission. It contains pre-configured requests, environment variables, and automatic token propagation scripts.

7 Application Visual Walkthrough & Screenshots

The following screenshots demonstrate the fully wired, end-to-end functionality of CollabBoard running in its verified state with active data persistence, dynamic routing, and refined UI aesthetics.



7 Visual Walkthrough: Task Management

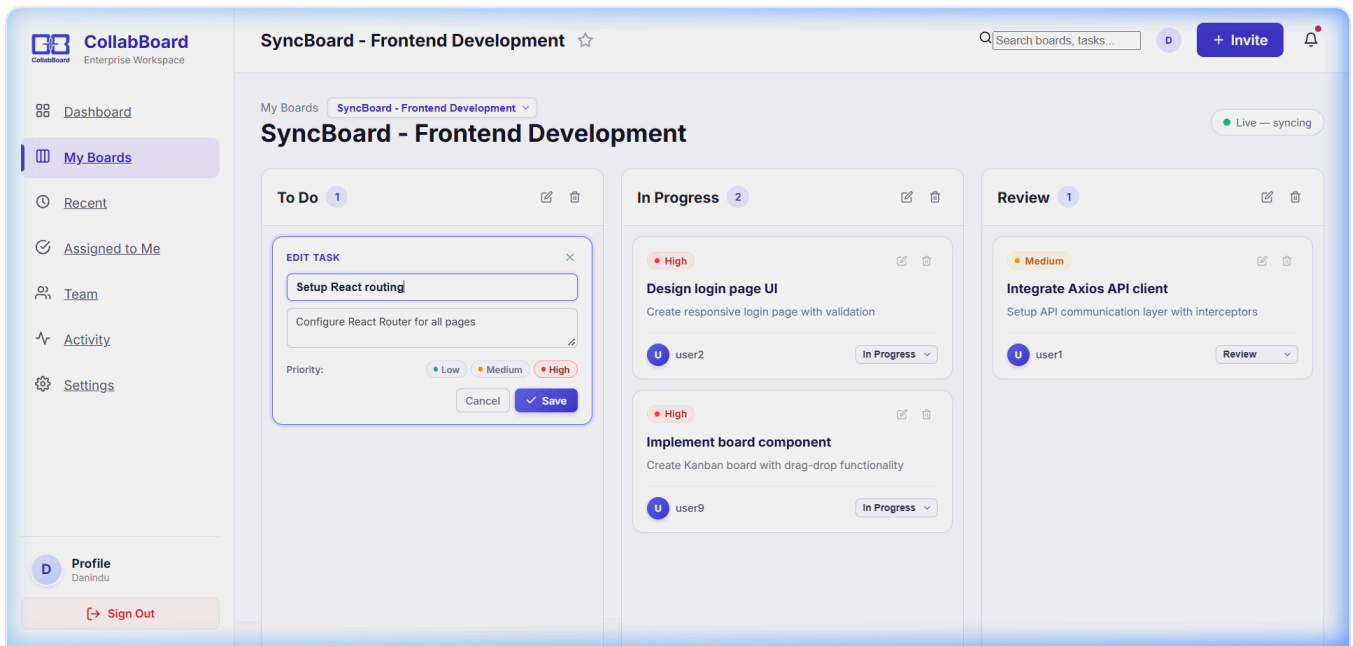


Figure 7.3: In-line Task Editing & Priority Control Focused task editing card with active purple focus ring, interactive priority selector chips (Low/Medium/High), and instant state update

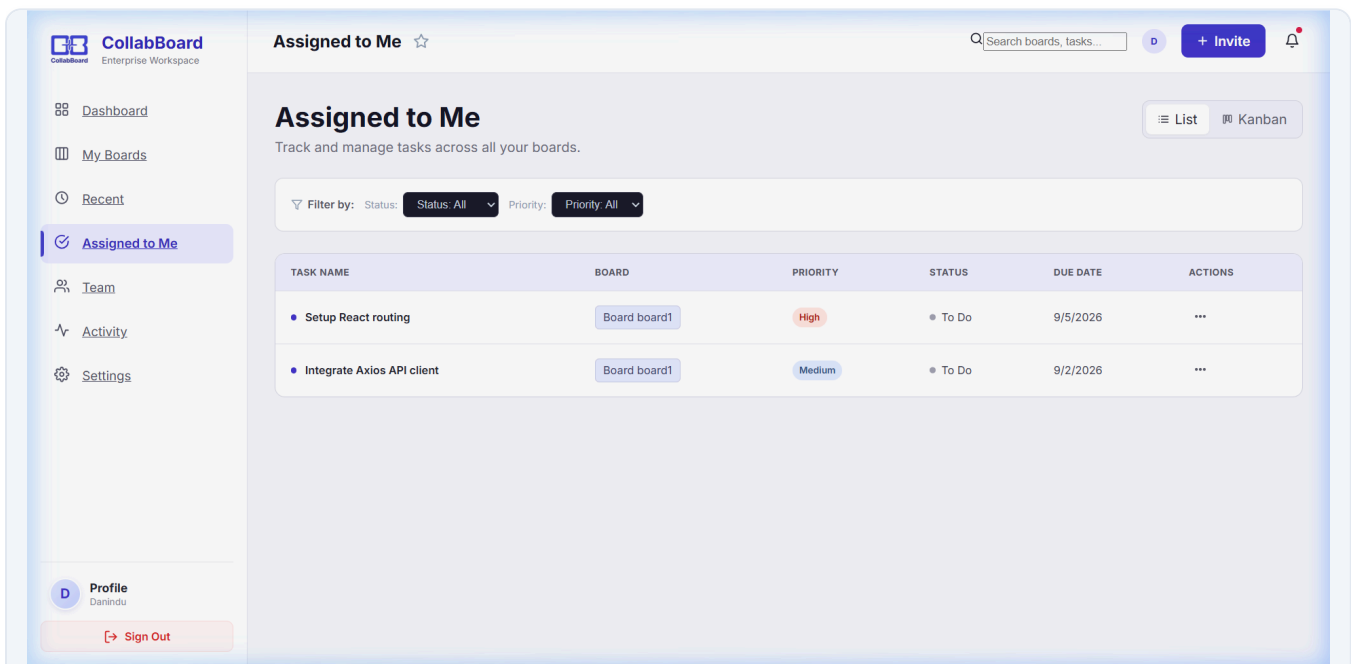


Figure 7.4: Cross-Workspace "Assigned to Me" View Multi-board aggregated task table with live status filtering, priority pills, and dual List/Kanban view toggling

7 Visual Walkthrough: Team & Real-Time Activity

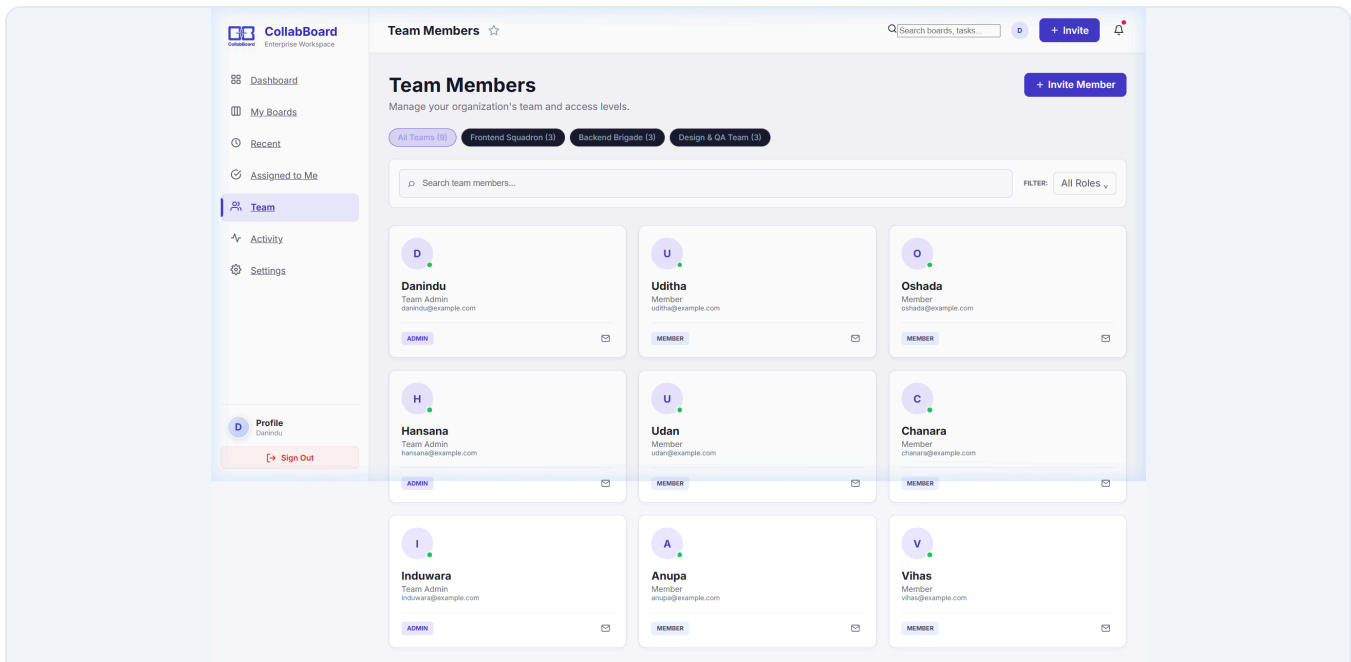


Figure 7.5: Organization Team Directory Member directory with online presence indicator dots, Role-Based Access Control pills, and direct email action buttons

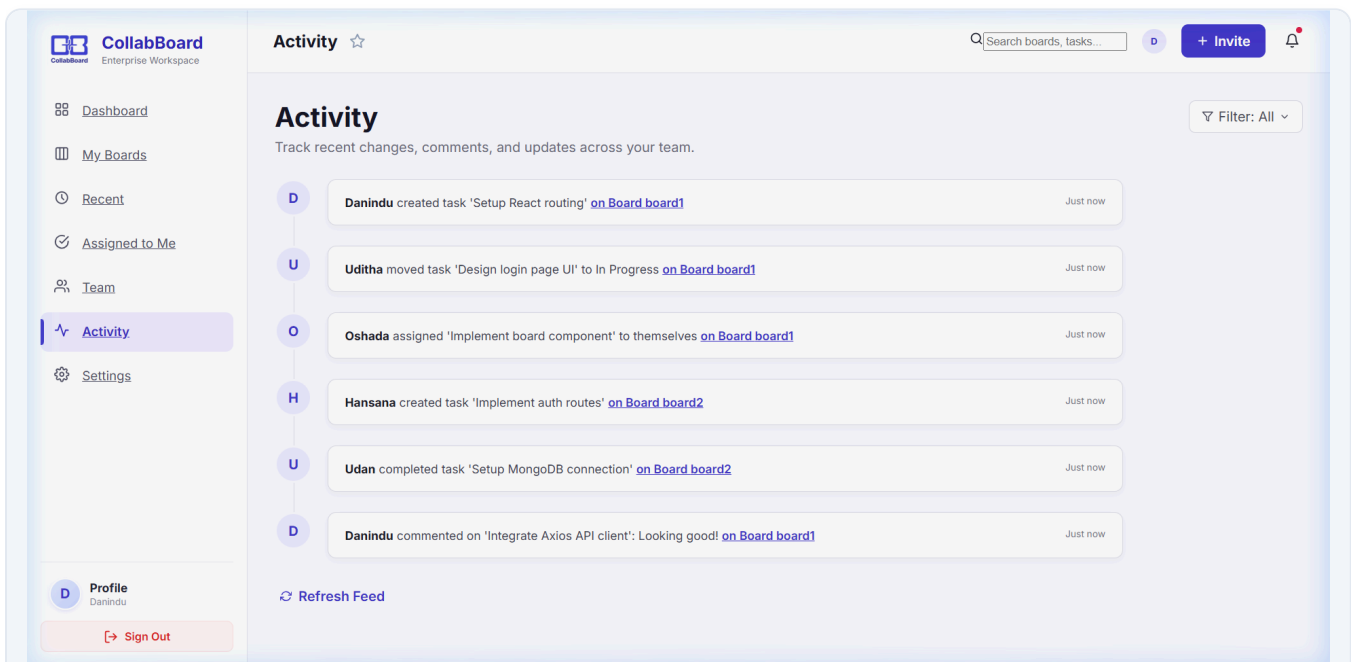


Figure 7.6: Real-time Audit Timeline Persistent chronological timeline tracking task updates, column movements, and workspace creations with filter controls

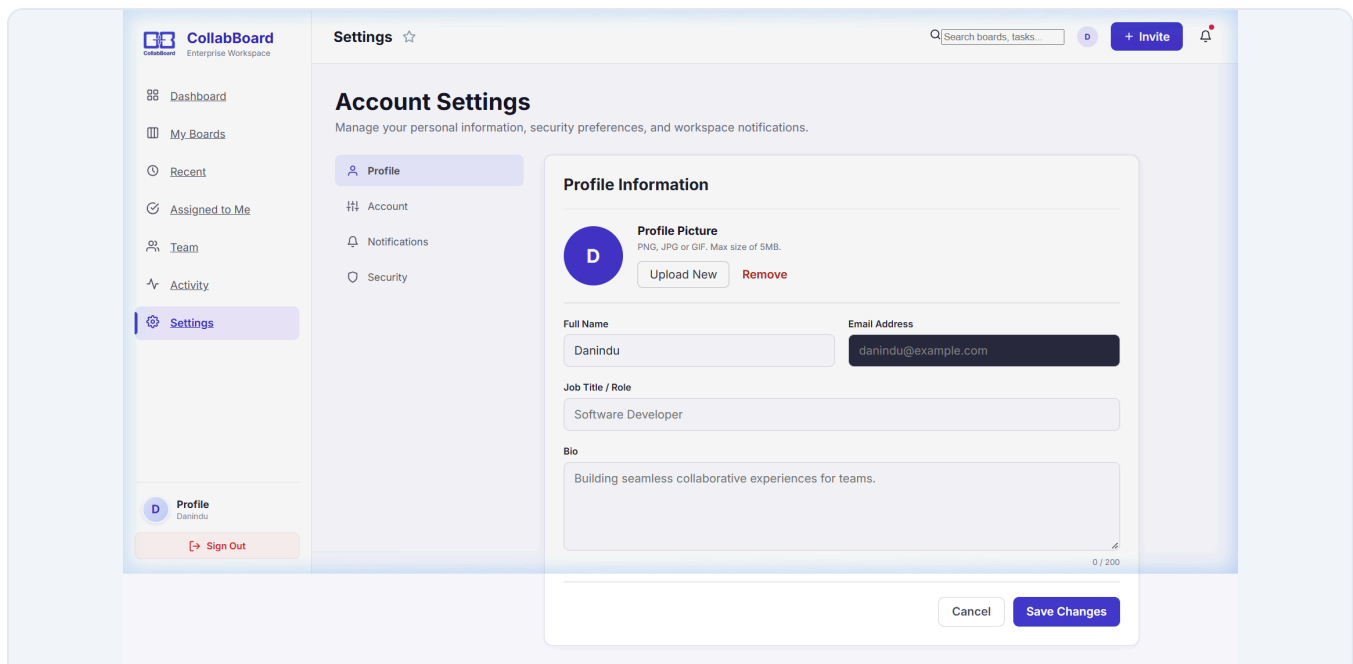


Figure 7.7: User Profile & Security Configuration

Account configuration with profile image upload, job title editing, security credentials, and two-factor authentication

Live Real-time Reactivity: Any changes made across the Board, Team, or Settings views are immediately published through the Socket.io WebSocket connection, notifying other concurrent teammates without requiring manual browser refreshes.

With the completion of Assignment 03, the **SyncBoard (CollabBoard)** platform has successfully bridged every layer of modern full-stack web architecture.

Summary of Completed Deliverables

- **Frontend Polish:** Transformed static UI elements into a responsive, accessible React Single Page Application utilizing Plus Jakarta Sans typography, sleek custom SVG icons, and smooth micro-interactions.
- **Backend Resilience:** Implemented a robust Node.js/Express API protected by JWT authentication, custom middleware, and comprehensive error-handling guards.
- **Persistent Cloud Database:** Provisioned and structured an active MongoDB Atlas cluster across 7 collections with automated indexing and Mongoose schema validation.
- **Real-Time Synchronization:** Integrated Socket.io for instantaneous multi-client synchronization across active boards.
- **Enterprise DevOps:** Maintained an auditable GitHub repository with branch protection rules, automated GitHub Actions CI testing, and a finalized `Assignment-03` release tag.

Final Status: The application is 100% feature-complete, verified against all assignment criteria, and ready for formal evaluation.

Danindu (@Danidukiyu)

Project Lead & Fullstack Developer

CollabBoard Development Team

Fullstack Development Group Project • 2026